

Laboratorio #2 – Análisis

Propiedades:

#	Tipo de dato	Nombre	Visibilidad	Motivo	Clase
1	String	codigo	private	Almacenar el identificador único del artista.	Artista
2	String	nombreArtistico	private	Almacenar el nombre con el que se presenta el artista.	Artista
3	String	generoMusical	private	Almacenar el estilo de música que interpreta el artista.	Artista
4	float	duracionMinutos	private	Guardar el tiempo de presentación, el cual debe validarse para ser mayor que 0.	Artista
5	int	cantidadAsistentes	private	Guardar la cantidad estimada de público, validando que no sea un valor negativo.	Artista
6	String	codigo	private	Almacenar el identificador único del escenario.	Escenario
7	String	nombre	private	Almacenar el nombre asignado al espacio físico.	Escenario
8	String	ubicacion	private	Guardar la ubicación del escenario dentro del festival.	Escenario
9	int	capacidadMaxima	private	Guardar el aforo máximo permitido, que obligatoriamente debe ser mayor que 0.	Escenario
10	String	estado	private	Indicar la situación actual del escenario.	Escenario

11	String	nombre	private	Almacenar el nombre oficial del festival.	Festival
12	String	codigo	private	Almacenar el código de identificación del evento.	Festival
13	String	coordinador	private	Guardar el nombre de la persona responsable de la organización.	Festival
14	Escenario[5]	escenarios	private	Arreglo básico con un límite fijo de 5 posiciones para administrar los espacios físicos disponibles.	Festival
15	ArrayList<Artista>	artistas	private	Colección dinámica que administra la cantidad variable de artistas; se debe usar ArrayList directamente y no List.	Festival
16	Scanner	scanner	private	Leer y capturar todos los datos numéricos y de texto ingresados por el usuario en la consola.	Vista
17	Vista	vista	private	Permitir al controlador invocar los menús, solicitar datos al usuario e imprimir resultados.	Controlador
18	Festival	festival	private	Permitir al controlador acceder a la lógica de negocio y modificar la información general, el arreglo y el ArrayList.	Controlador

Métodos

#	Tipo de dato	Nombre	Parámetros	Visibilidad	Motivo	Clase
1	Constructor	Artista	(String codigo, String nombreArtistico, String generoMusical, int duracionMinutos, int cantidadAsistentes)	public	Inicializar el objeto con los datos obligatorios.	Artista
2	String	getCodigo	()	public	Obtener el identificador del artista.	Artista
3	void	setCodigo	(String codigo)	public	Modificar el identificador del artista.	Artista
4	String	getNombreArtistico	()	public	Obtener el nombre artístico.	Artista
5	void	setNombreArtistico	(String nombreArtistico)	public	Modificar el nombre artístico.	Artista
6	String	getGeneroMusical	()	public	Obtener el género musical.	Artista
7	void	setGeneroMusical	(String generoMusical)	public	Modificar el género musical.	Artista
8	int	getDuracionMinutos	()	public	Obtener el tiempo de presentación.	Artista
9	void	setDuracionMinutos	(int duracionMinutos)	public	Modificar la duración, lanzando IllegalArgumentException si el valor es menor o igual a 0.	Artista
10	int	getCantidadAsistentes	()	public	Obtener la estimación de público.	Artista
11	void	setCantidadAsistentes	(int cantidadAsistentes)	public	Modificar los asistentes, lanzando IllegalArgumentException si el valor es negativo.	Artista
12	Constructor	Escenario	(String codigo, String nombre, String)	public	Inicializar el escenario con sus datos base.	Escenario

			ubicacion, int capacidadMaxi ma, String estado)			
1 3	String	getCodigo	()	public	Obtener el identificador del escenario.	Escenario
1 4	void	setCodigo	(String codigo)	public	Modificar el identificador del escenario.	Escenario
1 5	String	getNombre	()	public	Obtener el nombre del escenario.	Escenario
1 6	void	setNombre	(String nombre)	public	Modificar el nombre del escenario.	Escenario
1 7	String	getUbicaci on	()	public	Obtener la ubicación del escenario.	Escenario
1 8	void	setUbicaci on	(String ubicacion)	public	Modificar la ubicación del escenario.	Escenario
1 9	int	getCapaci dadMaxim a	()	public	Obtener el aforo máximo permitido.	Escenario
2 0	void	setCapacid adMaxima	(int capacidadMaxi ma)	public	Actualizar la capacidad, lanzando IllegalArgumentException si es menor o igual a 0.	Escenario
2 1	String	getEstado	()	public	Obtener la situación actual del escenario.	Escenario
2 2	void	setEstado	(String estado)	public	Modificar la situación actual del escenario.	Escenario
2 3	Const ructor	Festival	(String nombre, String codigo, String coordinador)	public	Inicializar el festival con sus datos principales.	Festival
2 4	String	getNombre	()	public	Obtener el nombre del festival.	Festival
2 5	void	setNombre	(String nombre)	public	Modificar el nombre del festival.	Festival
2 6	String	getCodigo	()	public	Obtener el código de identificación del evento.	Festival
2 7	void	setCodigo	(String codigo)	public	Modificar el código de identificación del evento.	Festival
2 8	String	getCoordin ador	()	public	Obtener el nombre de la persona responsable.	Festival
2 9	void	setCoordin ador	(String coordinador)	public	Modificar el nombre del responsable.	Festival

30	boolean / void	configurarEscenario	(int posicion, Escenario escenario)	public	Asignar un escenario en una posición del arreglo, validando que el índice exista y contenga null.	Festival
31	Escenario[]	consultarEscenarios	()	public	Retornar los escenarios del arreglo, omitiendo posiciones vacías.	Festival
32	Escenario	consultarEscenario	(int posicion)	public	Devolver el escenario de una posición específica.	Festival
33	void	modificarEscenario	(int posicion, int capacidad, String estado)	public	Actualizar un escenario; no se puede modificar una posición con null.	Festival
34	void	retirarEscenario	(int posicion)	public	Eliminar un escenario reasignando la posición seleccionada a null.	Festival
35	boolean / void	registrarArtista	(Artista artista)	public	Agregar un objeto al ArrayList, validando que no existan dos artistas con el mismo código.	Festival
36	ArrayList<Artista>	consultarArtistas	()	public	Devolver la colección dinámica de artistas registrados.	Festival
37	Artista	buscarArtista	(String codigo)	public	Recorrer el ArrayList y devolver el objeto del artista que coincida con el código.	Festival
38	void	cancelarParticipacion	(String codigo)	public	Localizar al artista por código y eliminarlo del ArrayList.	Festival
39	int	calcularEscenariosConfig	()	public	Contar cuántas posiciones del arreglo son diferentes de null.	Festival
40	int	calcularEspaciosDisponibles	()	public	Contar cuántas posiciones del arreglo contienen null.	Festival
41	Escenario	getEscenarioMayorCapacidad	()	public	Retornar el escenario con el	Festival

					valor más alto en capacidad máxima.	
4 2	Artista	getArtistaMayorDuracion	()	public	Devolver el artista con más minutos de presentación (solo si la lista no está vacía).	Festival
4 3	Artista	getArtistaMayorAsistentes	()	public	Devolver el artista con mayor estimación de público.	Festival
4 4	doble	calcularPromedioDuracion	()	public	Sumar las duraciones y dividir entre la cantidad registrada de artistas.	Festival
4 5	String	pedirTexto	(String mensaje)	public	Mostrar un mensaje y capturar un String desde la consola.	Vista
4 6	int	pedirEntero	(String mensaje)	public	Capturar un número, utilizando try-catch para limpiar si ocurre un InputMismatchException.	Vista
4 7	void	mostrarMenu	()	public	Imprimir en consola las 13 opciones requeridas.	Vista
4 8	void	mostrarMensaje	(String mensaje)	public	Imprimir reportes, confirmaciones o alertas de error sin que el programa finalice.	Vista
4 9	void	iniciar	()	public	Contener el ciclo del menú interactivo y el bloque finally para acciones finales.	Controlador

Preguntas

3. ¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo básico? ¿Qué tipo de objetos almacenará y cuál será su tamaño?

La propiedad que debe implementarse utilizando un arreglo básico es la colección de escenarios, la cual pertenece al festival. Este arreglo almacenará instancias de objetos de la clase Escenario y tendrá un tamaño máximo fijo de 5 posiciones.

4. ¿Cuál de las propiedades identificadas debe implementarse utilizando un ArrayList? ¿Qué tipo de objetos almacenará?

La propiedad que se debe implementar de forma obligatoria mediante un ArrayList es la colección dinámica de artistas participantes en el festival debido a que estos se pueden ir eliminando. Esta estructura almacenará objetos de la clase Artista.

7. ¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores deberán validarse antes de modificar el estado de los objetos?

Los valores iniciales se proveerán utilizando los constructores públicos de cada clase al momento de su instanciación. Antes de modificar el estado interno, se debe validar que la capacidad máxima de un escenario sea estrictamente mayor que 0. En el caso de los artistas, se debe validar que la duración de su presentación sea mayor que 0 y que la cantidad estimada de asistentes no sea un valor negativo.

8. ¿Cómo determinará si una posición del arreglo contiene un escenario o contiene null?

Se determinará evaluando directamente el índice específico del arreglo mediante una condición lógica. El programa realizará esta verificación antes de intentar acceder a la información o métodos de un escenario para evitar errores de referencias nulas.

9. ¿Cómo realizará las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?

Para las operaciones de búsqueda, se recorrerá el ArrayList empleando una estructura repetitiva para comparar el código ingresado por el usuario con el código almacenado en cada objeto registrado. Para la modificación, una vez localizado el artista, se utilizarán los métodos modificadores (setters) del objeto. Para la eliminación o cancelación de participación, se localizará al artista correspondiente y se removerá de la colección dinámica.

10. ¿Qué situaciones del programa pueden producir excepciones? Identifique qué excepciones deberán manejarse y en qué partes del programa utilizará try-catch y finally.

Las excepciones pueden ocurrir en dos situaciones concretas. Primero, se lanzará una `IllegalArgumentException` si se intentan asignar valores inválidos al crear o modificar objetos (capacidad de escenario menor o igual a 0, duración de artista menor o igual a 0, o cantidad de asistentes negativa). Segundo, ocurrirá una `InputMismatchException` si el usuario ingresa texto cuando el Scanner solicita un número. Para manejar esto, usaremos bloques try-catch exactamente al momento de leer los datos en la consola (para limpiar las entradas incorrectas) y al enviar la información a las clases, evitando así que el programa se cierre inesperadamente. Por último, el bloque finally se colocará en el ciclo del menú principal para asegurar la ejecución de instrucciones finales sin importar si ocurrieron errores o no.